MAIN.PY DOCUMENTATION
Biosensing Platform & ML Classification System
Piezo Pico-to-Femtotechnology Sensors Inc.
Version 1.0 — August 2026

Table of Contents

1. Introduction

This document provides a technical description of **main.py**, the central application file for the **Biosensing Platform & ML Classification System**. The main.py file initializes the FastAPI application, loads machine-learning models, mounts routers, configures middleware, and exposes documentation endpoints. The detailed information about main.py is attached as independent document, pdf file.

This document is intended for engineers, researchers, educators, and regulatory reviewers.

2. Purpose of main.py

The **main.py** file serves as the entry point for the backend API. Its responsibilities include:

- Creating the FastAPI application instance
- Loading ML models into memory
- Mounting all routers (simulation, classification, documentation, virus processing, training, etc.)
- Configuring middleware (CORS, logging, error handling)
- Providing metadata for Swagger/OpenAPI
- Starting the application server

main.py ensures that all components of the biosensing system operate cohesively.

3. System Initialization

main.py begins by importing:

- FastAPI
- Routers
- ML model loaders
- Utility modules
- CORS middleware
- System configuration

It then initializes the FastAPI application:

```
app = FastAPI(
    title="AI Biosensing API",
    version="1.0.0",
    description="Backend for Piezo Pico-to-Femtotechnology Sensors Inc."
)
```

This metadata appears in Swagger UI and OpenAPI JSON.

4. Router Mounting

main.py mounts all routers that define the API endpoints.

Core Biosensing Routers

- /simulate → simulate_router
- /classify/v2 → classify_v2_router
- /classify/v6 → classify_v6_router
- /classify/spectral → classify_spectral_router
- /docs → docs_router

Extended Routers (from Swagger):

- Authentication
- Payments
- Courses & Modules
- Course Content
- Enrollment
- Consultations
- Students & Teams
- Virus Processing
- Model Versioning
- ML Training
- Sensor Drift

Routers are mounted using:

- app.include_router(simulate_router)
- app.include_router(classify_v2_router)
- app.include_router(classify_v6_router)
- app.include_router(classify_spectral_router)
- app.include_router(docs_router)

5. ML Model Loading

main.py loads ML models at startup so they remain in memory for fast inference.

Models are stored in:

- ml/models/classify/v2/sim_model_v2_100.pkl
- ml/models/classify/v6/sim_model_v6_100.pkl
- ml/models/classify/spectral/spectral_model.pkl

Loading occurs as:

- model_v2 = joblib.load(MODEL_V2_PATH)
- model_v6 = joblib.load(MODEL_V6_PATH)
- spectral_model = joblib.load(MODEL_SPECTRAL_PATH)

This ensures:

- Fast response times
- No repeated disk access
- Consistent model state

If a model fails to load, main.py logs the error and prevents the API from starting.

6. Middleware & CORS

main.py configures CORS to allow frontend dashboards to communicate with the backend:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"]
)
```

This enables:

- Local development
- Cloud deployment
- Cross-domain dashboard access

7. Documentation Endpoints

main.py exposes:

Swagger UI Through:

- http://127.0.0.1:8000/docs

OpenAPI JSON Through:

- http://127.0.0.1:8000/openapi.json

Custom Documentation Router

The docs_router provides:

- Links to BIOSENSING documentation
- Router documentation
- Folder structure documentation
- ML model documentation
- Device documentation

This ensures all technical documents are accessible through the API.

8. Error Handling

main.py includes global exception handlers for:

- Validation errors
- ML inference errors
- Missing data
- Internal server errors

Errors are logged with timestamps and stack traces for debugging.

9. Execution & Deployment

Local Development

Run the API using:

uvicorn app.main:app –reload or uvicorn app.main:app --reload --host 127.0.0.1 --port 8000

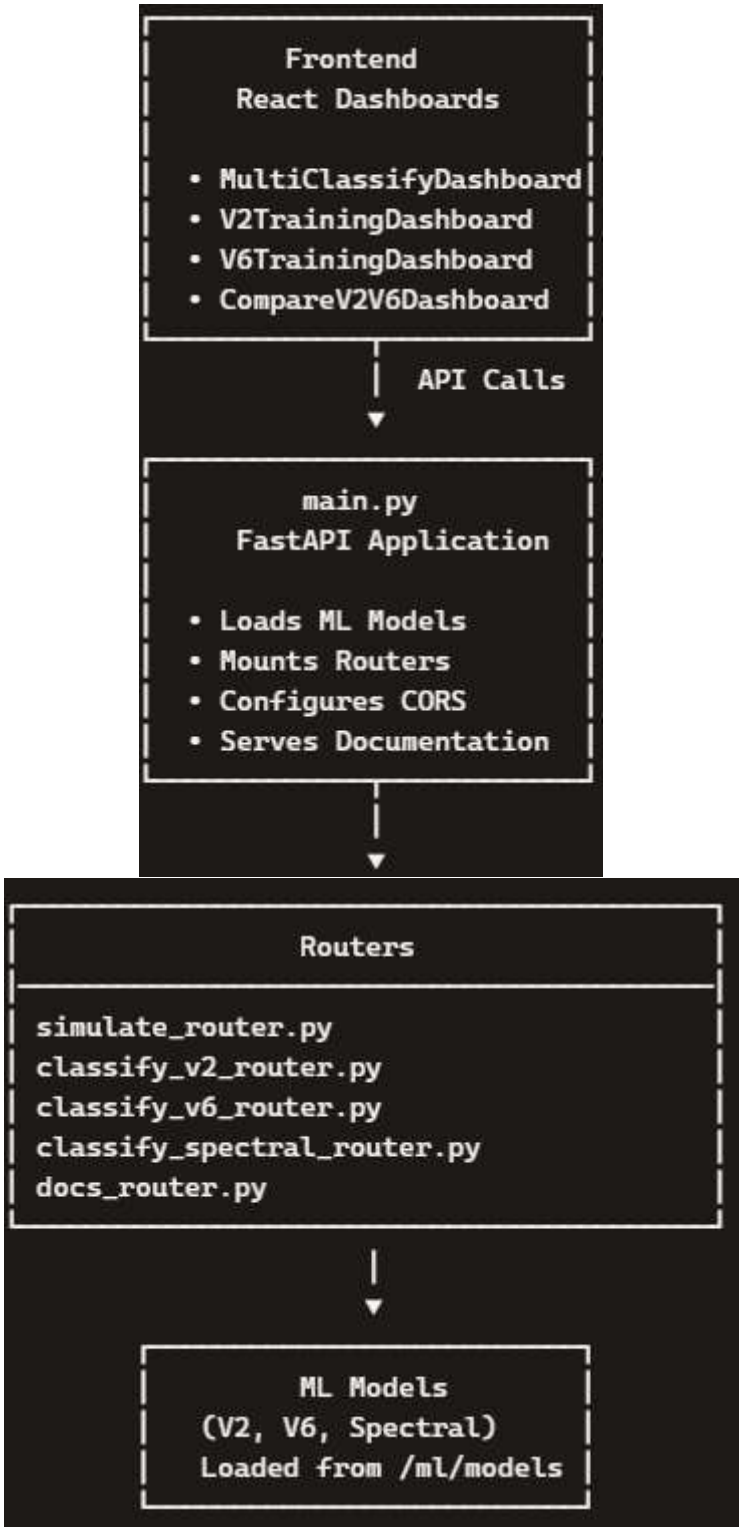
Production Deployment

Recommended:

- Uvicorn + Gunicorn
- Nginx reverse proxy
- HTTPS enabled
- Model caching enabled
- Logging to cloud storage

10. Architecture Diagram

See next page


Frontend
React Dashboards
• MultiClassifyDashboard
• V2TrainingDashboard
• V6TrainingDashboard
• CompareV2V6Dashboard
API Calls
main.py
FastAPI Application
• Loads ML Models
• Mounts Routers
• Configures CORS
• Serves Documentation
Routers
simulate_router.py
classify_v2_router.py
classify_v6_router.py
classify_spectral_router.py
docs_router.py
ML Models
(V2, V6, Spectral)
Loaded from /ml/models

11. Publication & Contact Information

Publisher:

Piezo Pico-to-Femtotechnology Sensors Inc. Edmonton, Alberta, Canada

Contact:

- **Email:** selemaniseif12@yahoo.com or selemaniseif1974@gmail.com
- **Website:** www.pico-femto-sensors.com
- **Support:** selemaniseif12@yahoo.com

Copyright

12. Conclusion

This main.py documentation completes the backend technical reference for the biosensing system. It explains how the application initializes, loads models, mounts routers, and exposes documentation endpoints.

This API system is now fully documented and ready for:

- Publication
- Deployment
- Regulatory review
- Educational use
- Trial research review